LAB 2

Cel: Stworzyć prosty symulator do zarządzania siecią sklepów z używanymi laptopami. Sieć składa się z oddziałów (punktów sprzedaży), a w każdym oddziale znajdują się laptopy przyjęte do odsprzedaży.

Przygotowanie:

1. Typ wyliczeniowy LaptopCondition z wartościami: Sprawny / Uszkodzony / W naprawie / W trakcie testów (laptop przyjęty do sklepu, którego stan nie został jeszcze zweryfikowany).
2. Klasę Laptop z polami: model (String), brand (String), condition (LaptopCondition), year (int - rok produkcji), price (double - cena w zł). Zaproponuj inne pola (np. ramGb, processor, batteryHealth - kondycja baterii w %, serialNumber).
    - Konstruktor pozwalający na łatwą inicjalizację obiektu (powyższe pola).
    - Metodę print() wypisującą na standardowe wyjście pełne informacje o laptopie.
    - Klasa Laptop powinna implementować interfejs Comparable<Laptop>, który pozwala na porównanie laptopów według: modelu, marki, wieku.
3. Klasę Branch (oddział sklepu), która zawiera takie informacje jak: branchName (String), laptopList (List<Laptop>), maxCapacity (int - maksymalna liczba laptopów, które można przechowywać w oddziale) oraz następujące metody:
    - addLaptop(Laptop laptop) - dodająca laptop do oddziału. Jeśli dany laptop już istnieje (porównując według modelu, marki i wieku), wyświetl komunikat. Jeśli pojemność zostanie przekroczona, wypisz komunikat na standardowe wyjście błędów (System.err).
    - removeLaptop(Laptop laptop) - usuwająca laptop z oddziału (np. utylizacja, zwrot do dostawcy).
    - sellLaptop(Laptop laptop) - usuwająca laptop z oddziału w wyniku sprzedaży (lub przeniesienia do innego oddziału).
    - changeCondition(Laptop laptop, LaptopCondition condition) - zmieniająca stan laptopa.
    - changeYear(Laptop laptop, int year) - zmieniająca rok produkcji laptopa (w przypadku błędnie wpisanego podczas przyjęcia).
    - countByCondition(LaptopCondition condition) - zwracająca liczbę laptopów w danym stanie.
    - sortByName() - zwracająca listę laptopów posortowaną alfabetycznie po modelu.
    - sortByPrice() - zwracająca listę laptopów posortowaną po cenie rosnąco. Zastosuj Comparator.
    - search(String model) - przyjmująca nazwę modelu i zwracająca laptop. Zastosuj Comparator.
    - searchPartial(String text) - przyjmująca fragment nazwy modelu lub marki i zwracająca wszystkie laptopy, które pasują.
    - summary() - wypisująca na standardowe wyjście informacje o wszystkich laptopach.
    - max() - zwracająca najdroższy laptop; zastosuj metodę Collections.max.
4. Klasę StoreManager, która będzie przechowywała oddziały w Map<String, Branch> (kluczem jest nazwa oddziału). Zaimplementuj metody:
    - addBranch(String name, int capacity) - dodająca nowy oddział o podanej nazwie i zadanej pojemności.
    - removeBranch(String name) - usuwająca oddział o podanej nazwie.
    - findEmpty() - zwracająca listę pustych oddziałów.
    - summary() - wypisująca na standardowe wyjście informacje zawierające: nazwę oddziału i jego procentowe zapełnienie.

Dodaj inne przydatne metody i zmienne (np. transferLaptop(Laptop laptop, String fromBranch, String toBranch) przenoszące laptop między oddziałami, totalValue() zwracające łączną wartość laptopów w oddziale).

Pokaż działanie wszystkich metod w aplikacji w metodzie main, uruchamiając każdą metodę według potrzeb. Nie trzeba tworzyć menu - wystarczą przykładowe wywołania metod.

Wskazówki

1. Typ wyliczeniowy z automatyczną konwersją na String

```
public enum LaptopCondition {
  WORKING {
    @Override
    public String toString() {
      return "Sprawny";
    }
  },
  DAMAGED,
  IN_REPAIR,
  IN_TESTING
}
```

2. Jak wykorzystać Comparator w algorytmach:

```
List<Laptop> laptops = new ArrayList<>();
laptops.add(new Laptop("ThinkPad T480", "Lenovo", LaptopCondition.WORKING, 6, 1199.0));
laptops.add(new Laptop("Latitude 7490", "Dell", LaptopCondition.IN_TESTING, 6, 999.0));
// Implementacja in-place - klasa anonimowa
Laptop l1 = Collections.max(laptops, new Comparator<Laptop>() {
  @Override
  public int compare(Laptop o1, Laptop o2) {
    return Double.compare(o1.getPrice(), o2.getPrice());
  }
});
// Implementacja przez wyrażenie lambda
Laptop l2 = Collections.max(laptops, (o1, o2) -> {
  return Double.compare(o1.getPrice(), o2.getPrice());
});
// Referencja do metody
Laptop l3 = Collections.max(laptops, Comparator.comparingDouble(Laptop::getPrice));
```

https://javastart.pl/static/algorytmy/sortowanie-kolekcji-interfejsy-comparator-i-comparable/

3. Metoda contains(String) klasy String zwraca true jeśli podany w argumencie napis zawiera się w obiekcie na rzecz którego została uruchomiona metoda. https://www.tutorialspoint.com/java/lang/string_contains.htm
4. Interfejsy Comparable oraz Comparator są częścią języka Java! Implementując metodę compareTo lub compare pamiętaj, że muszą one zwracać liczbę całkowitą. Jeśli obiekt ma być w pewnej hierarchii przed innym to zwracamy wartość mniejszą od 0, jeśli za innym to większą od 0, natomiast jeśli są równe to zwracane jest 0. Metodę compareTo możesz jawnie uruchomić np. na obiekcie typu String w celu jego porównania

Teoria

1. Co zyskujemy pisząc List<?> myList = new ArrayList<?>(); zamiast ArrayList<?> myList = new ArrayList<?>();
2. ArrayList vs LinkedList - kiedy używać jakich list? https://javastart.pl/static/klasy/interfejs-list/
3. HashMap vs TreeMap vs LinkedHashMap - kiedy używać jakich map https://javastart.pl/static/klasy/interfejs-map/
4. List vs Map vs Set - w jakich przypadkach użyć którą kolekcję?
5. Interfejs Comparable - jak go używać? jakie problemy rozwiązuje?
6. Użyteczne metody algorytmiczne z klasy Collections (sort, max)
7. Różnica między metodą equals a operatorem == (na przykładzie obiektu String)
8. Po co używamy adnotacji @Override https://stackoverflow.com/questions/94361/when-do-you-use-javas-override-annotation-andwhy
9. Klasa wewnętrzna i anonimowa klasa wewnętrzna (anonymous inner class). Gdzie i po co je wykorzystujemy (odpowiedzieć na przykładach).
10. Czym są wyrażenia lambda, jak się je konstruuje, gdzie mogą być przydatne.

Teoria OBOWIĄZUJE we wszystkich programach (również na kolejnych zajęciach).

Po uzyskaniu zaliczenia na zajęciach, prześlij źródła w archiwum zgodnie z konwencją nazewniczą.